Analytical Problem Solving:

Delivery Robot in a 5×5 Warehouse

Problem Setup

Cells are written as (row, column). The robot starts at S = (1,1) and must reach G = (5,5). Blocked cells (shelving units, marked X) are (2,2), (2,3), (3,3), (4,2) and (4,4). Moves: Up = row − 1, Down = row + 1, Left = column − 1, Right = column + 1; every move costs 1.

	c1	c2	c3	c4	c5
r1	S				
r2		X	X		
r3			X		
r4		X		X	
r5					G

Tie-breaking convention used in all traces: neighbours are generated in the order Up, Down, Left, Right. Both searches use a visited/explored set so that no cell is entered twice.

a. PEAS Description

Component	Description
Performance measure	Deliver the package at (5,5) (task success). Minimise path cost, i.e. the number of moves (time/energy). Never enter a blocked cell or leave the grid (safety, no collisions).
Environment	5×5 discrete grid warehouse with 5 blocked cells; 20 free cells. Fully observable and known map, deterministic, static (shelves do not move), single agent, sequential, discrete.
Actuators	Drive motors/wheels for the four moves: Up, Down, Left, Right (one cell each). Unloading/release mechanism to drop the package at the goal.
Sensors	Position sensor (wheel encoders, floor markers or camera) giving the current cell. Proximity/obstacle sensors (LiDAR, ultrasonic) to confirm a neighbouring cell is blocked. Package-presence sensor.

Input:

PEAS – Warehouse Robot.py - C:\Users\Anusha\AppData\Local\Programs\Python\Python311\PEAS – Warehouse Robot.py (3.11.2)

File Edit Format Run Options Window Help

```
print("PEAS Description of Warehouse Robot")

print("\nPerformance Measure:")
print("Deliver the package to the destination using the shortest path.")

print("\nEnvironment:")
print("5 x 5 warehouse grid with blocked cells.")

print("\nActuators:")
print("Move Up, Down, Left or Right.")

print("\nSensors:")
print("Detect current position, obstacles and destination.")
```

Output:

IDLE Shell 3.11.2

File Edit Shell Debug Options Window Help

```
Python 3.11.2 (tags/v3.11.2:878ead1, Feb  7 2023, 16:38:35) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Anusha\AppData\Local\Programs\Python\Python311\PEAS - Warehouse Robot.py
PEAS Description of Warehouse Robot

Performance Measure:
Deliver the package to the destination using the shortest path.

Environment:
5 x 5 warehouse grid with blocked cells.

Actuators:
Move Up, Down, Left or Right.

Sensors:
Detect current position, obstacles and destination.
>>>
```

b. Type of Agent: Goal-Based Agent

A **goal-based agent** (that also keeps a model of the map) is the best fit, for these reasons:

- **Simple reflex is unsuitable:** it reacts only to the current cell. The map contains dead ends, for example (3,2) and (4,3), which a reflex rule such as “move toward (5,5)” would walk into and could oscillate between.
- **Model-based alone is not enough:** it tracks the state and knows what the actions do, but has no explicit target to decide which action is better.
- **Goal-based fits:** the task has one explicit goal, (5,5). The agent knows the map and the effect of each move, so it can search ahead, find a complete sequence of moves, and avoid dead ends before moving.
- **Utility-based is unnecessary:** all moves cost 1 and there is a single objective, so there is no trade-off (risk, energy, congestion) to weigh. It would become appropriate if aisles had different costs or the robot had to balance speed against safety.

Input:

Goal-Based Agent.py - C:\Users\Anusha\AppData\Local\Programs\Python\Python311\Goal-Based Agent.py (3.11.2)

File Edit Format Run Options Window Help

```
start = (1, 1)
goal = (5, 5)

print("Agent Type: Goal-Based Agent")
print("Start Position:", start)
print("Goal Position:", goal)

if start != goal:
    print("Robot will select actions to reach the goal.")
else:
    print("Goal reached.")
```

Output:

IDLE Shell 3.11.2

File Edit Shell Debug Options Window Help

```
Python 3.11.2 (tags/v3.11.2:878ead1, Feb  7 2023, 16:38:35) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Anusha\AppData\Local\Programs\Python\Python311\Goal-Based Agent.py
Agent Type: Goal-Based Agent
Start Position: (1, 1)
Goal Position: (5, 5)
Robot will select actions to reach the goal.
>>>
```

c. Formal Search Problem Definition

Element	Definition
State space	All cells (r, c) with 1 ≤ r, c ≤ 5 that are not blocked (20 states).
Initial state	(1,1)
Goal state	(5,5). Goal test: state = (5,5).
Actions	ACTIONS(r,c) = the subset of {Up, Down, Left, Right} whose resulting cell is inside the grid and not blocked.
Transition model	RESULT((r,c), Up) = (r−1, c) RESULT((r,c), Down) = (r+1, c) RESULT((r,c), Left) = (r, c−1) RESULT((r,c), Right) = (r, c+1) (only if the resulting cell is legal; otherwise the action is not available)
Path cost	Step cost c(s, a, s′) = 1 for every move, so g(path) = number of moves.

Input:

search problem formation.py - C:\Users\Anusha\AppData\Local\Programs\Python\Python311\search problem formation.py (3.11.2)

File Edit Format Run Options Window Help

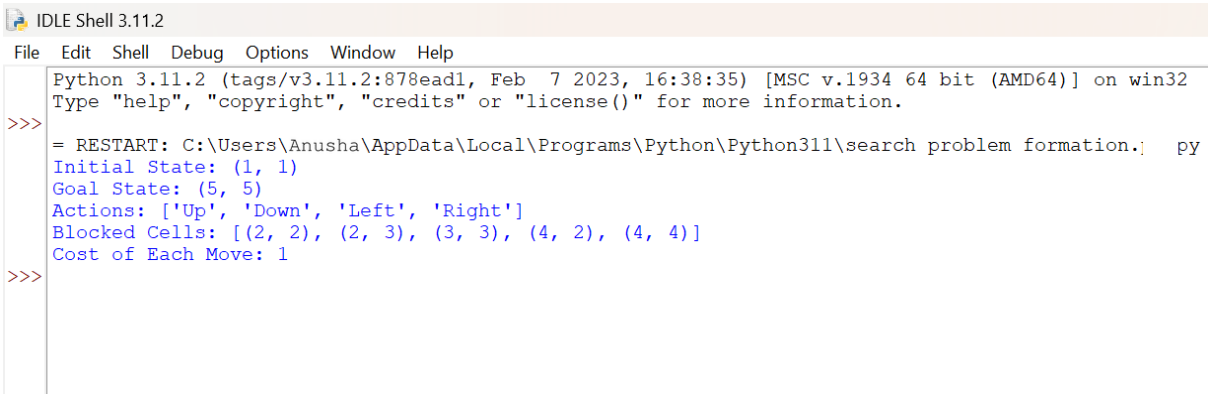
```
start = (1, 1)
goal = (5, 5)

blocked = [(2, 2), (2, 3), (3, 3), (4, 2), (4, 4)]

actions = ["Up", "Down", "Left", "Right"]
cost = 1

print("Initial State:", start)
print("Goal State:", goal)
print("Actions:", actions)
print("Blocked Cells:", blocked)
print("Cost of Each Move:", cost)
```

Output:


IDLE Shell 3.11.2
File Edit Shell Debug Options Window Help
Python 3.11.2 (tags/v3.11.2:878ead1, Feb 7 2023, 16:38:35) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Anusha\AppData\Local\Programs\Python\Python311\search problem formation.py
Initial State: (1, 1)
Goal State: (5, 5)
Actions: ['Up', 'Down', 'Left', 'Right']
Blocked Cells: [(2, 2), (2, 3), (3, 3), (4, 2), (4, 4)]
Cost of Each Move: 1
>>>

d. Breadth-First Search (BFS)

BFS uses a FIFO queue. The goal test is applied when a node is generated. "Children added" lists only unvisited cells. Frontier is shown front → back.

Step	Node expanded	Children added	Frontier (queue) after step
0	–	–	(1,1)
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1)	(1,2), (3,1)
3	(1,2)	(1,3)	(3,1), (1,3)
4	(3,1)	(4,1), (3,2)	(1,3), (4,1), (3,2)
5	(1,3)	(1,4)	(4,1), (3,2), (1,4)
6	(4,1)	(5,1)	(3,2), (1,4), (5,1)
7	(3,2)	none (dead end)	(1,4), (5,1)
8	(1,4)	(2,4), (1,5)	(5,1), (2,4), (1,5)
9	(5,1)	(5,2)	(2,4), (1,5), (5,2)
10	(2,4)	(3,4), (2,5)	(1,5), (5,2), (3,4), (2,5)
11	(1,5)	none ((2,5) already queued)	(5,2), (3,4), (2,5)
12	(5,2)	(5,3)	(3,4), (2,5), (5,3)
13	(3,4)	(3,5)	(2,5), (5,3), (3,5)
14	(2,5)	none ((3,5) already queued)	(5,3), (3,5)
15	(5,3)	(4,3), (5,4)	(3,5), (4,3), (5,4)
16	(3,5)	(4,5)	(4,3), (5,4), (4,5)
17	(4,3)	none (dead end)	(5,4), (4,5)
18	(5,4)	(5,5) = **GOAL**	Goal generated, stop

Result: Path = (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5). **Total cost = 8** (8 moves, depth 8). Nodes expanded = 18; maximum frontier size = 4.

Other cost-8 paths exist (for example along the top row and right column); which one BFS returns depends only on the tie-breaking order. BFS is guaranteed to return a cost-8 path, which is the minimum because the Manhattan distance from (1,1) to (5,5) is 8.

input:

bfs program.py - C:\Users\Anusha\AppData\Local\Programs\Python\Python311\bfs program.py (3.11.2)

File Edit Format Run Options Window Help

```
from collections import deque

grid_size = 5

start = (1, 1)
goal = (5, 5)

blocked = {
    (2, 2), (2, 3), (3, 3),
    (4, 2), (4, 4)
}

moves = [
    (-1, 0),    # Up
    (1, 0),     # Down
    (0, -1),    # Left
    (0, 1)      # Right
]

queue = deque()
queue.append((start, [start]))

visited = {start}

while queue:

    current, path = queue.popleft()

    print("Expanded:", current)
    print("Queue:", list(queue))

    if current == goal:
        print("\nGoal Reached!")
        print("BFS Path:", path)
        print("Total Cost:", len(path) - 1)
        break

    for move in moves:

        new_row = current[0] + move[0]
        new_col = current[1] + move[1]
```

bfs program.py - C:\Users\Anusha\AppData\Local\Programs\Python\Python311\bfs program.py (3.11.2)

File Edit Format Run Options Window Help

```
    (1, 0),     # Down
    (0, -1),    # Left
    (0, 1)      # Right
]

queue = deque()
queue.append((start, [start]))

visited = {start}

while queue:

    current, path = queue.popleft()

    print("Expanded:", current)
    print("Queue:", list(queue))

    if current == goal:
        print("\nGoal Reached!")
        print("BFS Path:", path)
        print("Total Cost:", len(path) - 1)
        break

    for move in moves:

        new_row = current[0] + move[0]
        new_col = current[1] + move[1]

        new_position = (new_row, new_col)

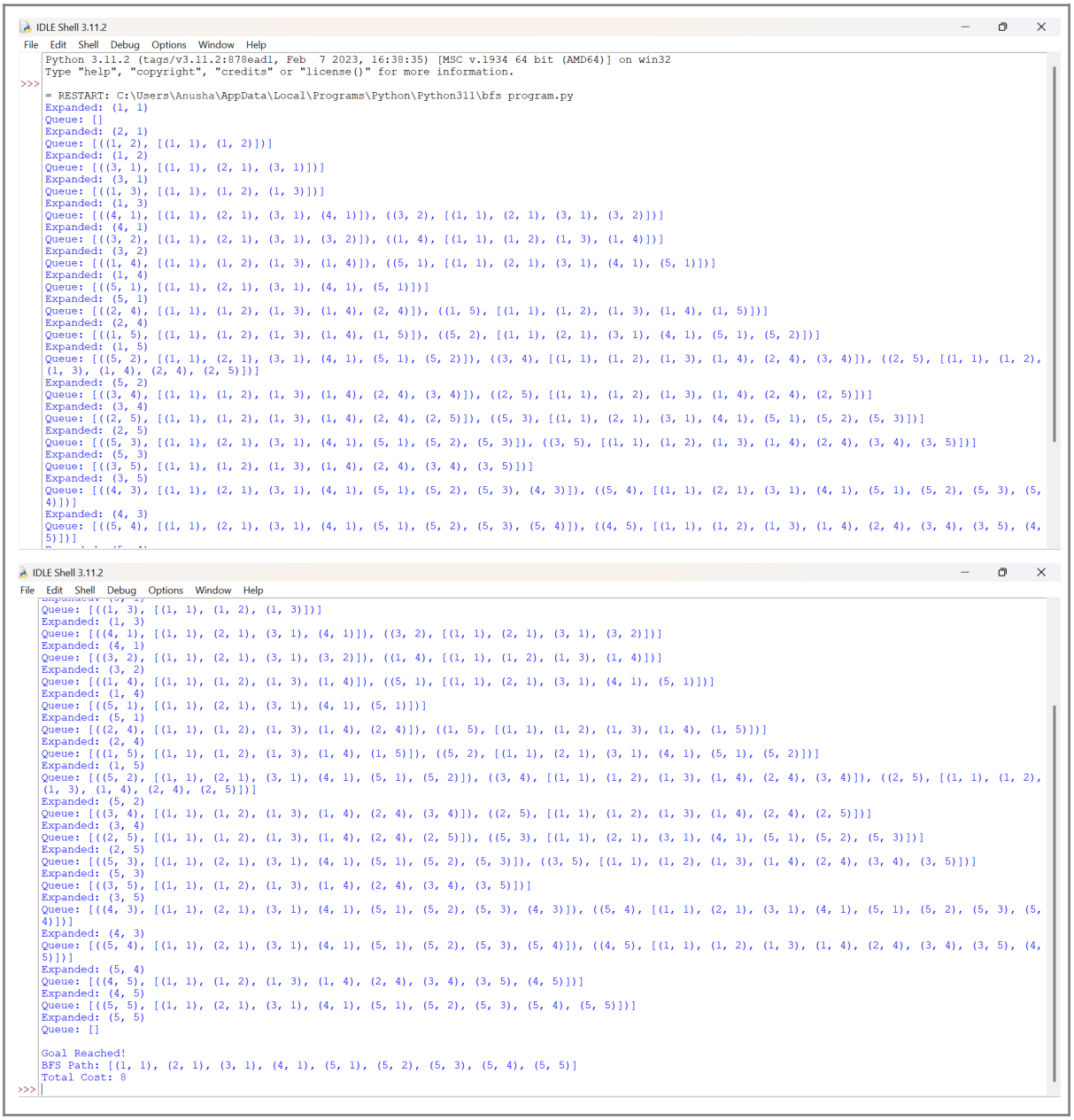
        if (1 <= new_row <= 5 and
            1 <= new_col <= 5 and
            new_position not in blocked and
            new_position not in visited):

            visited.add(new_position)

            new_path = path + [new_position]

            queue.append((new_position, new_path))
```

Output (BFS):


IDLE Shell 3.11.2
File Edit Shell Debug Options Window Help
Python 3.11.2 (tags/v3.11.2:878ead1, Feb 7 2023, 16:38:35) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Anusha\AppData\Local\Programs\Python\Python311\bfs program.py
Expanded: (1, 1)
Queue: []
Expanded: (2, 1)
Queue: [((1, 2), [(1, 1), (1, 2)])]
Expanded: (1, 2)
Queue: [((3, 1), [(1, 1), (2, 1), (3, 1)])]
Expanded: (3, 1)
Queue: [((1, 3), [(1, 1), (1, 2), (1, 3)])]
Expanded: (1, 3)
Queue: [((4, 1), [(1, 1), (2, 1), (3, 1), (4, 1)]), ((3, 2), [(1, 1), (2, 1), (3, 1), (3, 2)])]
Expanded: (4, 1)
Queue: [((3, 2), [(1, 1), (2, 1), (3, 1), (3, 2)]), ((1, 4), [(1, 1), (1, 2), (1, 3), (1, 4)])]
Expanded: (3, 2)
Queue: [((1, 4), [(1, 1), (1, 2), (1, 3), (1, 4)]), ((5, 1), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1)])]
Expanded: (1, 4)
Queue: [((5, 1), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1)])]
Expanded: (5, 1)
Queue: [((2, 4), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4)]), ((1, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5)])]
Expanded: (2, 4)
Queue: [((1, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5)]), ((5, 2), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2)])]
Expanded: (1, 5)
Queue: [((5, 2), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2)]), ((3, 4), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4)]), ((2, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (2, 5)])]
Expanded: (5, 2)
Queue: [((3, 4), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4)]), ((2, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (2, 5)])]
Expanded: (3, 4)
Queue: [((2, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (2, 5)]), ((5, 3), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3)])]
Expanded: (2, 5)
Queue: [((5, 3), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3)]), ((3, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (3, 5)])]
Expanded: (5, 3)
Queue: [((3, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (3, 5)])]
Expanded: (3, 5)
Queue: [((4, 3), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (4, 3)]), ((5, 4), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4)])]
Expanded: (4, 3)
Queue: [((5, 4), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4)]), ((4, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (3, 5), (4, 5)])]
IDLE Shell 3.11.2
File Edit Shell Debug Options Window Help
Queue: [((1, 3), [(1, 1), (1, 2), (1, 3)])]
Expanded: (1, 3)
Queue: [((4, 1), [(1, 1), (2, 1), (3, 1), (4, 1)]), ((3, 2), [(1, 1), (2, 1), (3, 1), (3, 2)])]
Expanded: (4, 1)
Queue: [((3, 2), [(1, 1), (2, 1), (3, 1), (3, 2)]), ((1, 4), [(1, 1), (1, 2), (1, 3), (1, 4)])]
Expanded: (3, 2)
Queue: [((1, 4), [(1, 1), (1, 2), (1, 3), (1, 4)]), ((5, 1), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1)])]
Expanded: (1, 4)
Queue: [((5, 1), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1)])]
Expanded: (5, 1)
Queue: [((2, 4), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4)]), ((1, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5)])]
Expanded: (2, 4)
Queue: [((1, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (1, 5)]), ((5, 2), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2)])]
Expanded: (1, 5)
Queue: [((5, 2), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2)]), ((3, 4), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4)]), ((2, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (2, 5)])]
Expanded: (5, 2)
Queue: [((3, 4), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4)]), ((2, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (2, 5)])]
Expanded: (3, 4)
Queue: [((2, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (2, 5)]), ((5, 3), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3)])]
Expanded: (2, 5)
Queue: [((5, 3), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3)]), ((3, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (3, 5)])]
Expanded: (5, 3)
Queue: [((3, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (3, 5)])]
Expanded: (3, 5)
Queue: [((4, 3), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (4, 3)]), ((5, 4), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4)])]
Expanded: (4, 3)
Queue: [((5, 4), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4)]), ((4, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (3, 5), (4, 5)])]
Expanded: (5, 4)
Queue: [((4, 5), [(1, 1), (1, 2), (1, 3), (1, 4), (2, 4), (3, 4), (3, 5), (4, 5)])]
Expanded: (4, 5)
Queue: [((5, 5), [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)])]
Expanded: (5, 5)
Queue: []
Goal Reached!
BFS Path: [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
Total Cost: 8
>>>

e. Depth-First Search (DFS)

DFS uses a LIFO stack, with an explored set. Neighbours are pushed so that the first one in the order Up, Down, Left, Right is on top and is popped first. The goal test is applied when a node is popped. The stack is shown top → bottom.

Step	Node popped	Children pushed	Stack after step (top → bottom)
0	–	–	(1,1)
1	(1,1)	(2,1), (1,2)	(2,1), (1,2)
2	(2,1)	(3,1)	(3,1), (1,2)
3	(3,1)	(4,1), (3,2)	(4,1), (3,2), (1,2)
4	(4,1)	(5,1)	(5,1), (3,2), (1,2)
5	(5,1)	(5,2)	(5,2), (3,2), (1,2)
6	(5,2)	(5,3)	(5,3), (3,2), (1,2)
7	(5,3)	(4,3), (5,4)	(4,3), (5,4), (3,2), (1,2)
8	(4,3)	none (dead end, backtrack)	(5,4), (3,2), (1,2)
9	(5,4)	(5,5)	(5,5), (3,2), (1,2)
10	(5,5) = **GOAL**	–	Goal popped, stop ((3,2), (1,2) left unexplored)

Result: Path = (1,1) → (2,1) → (3,1) → (4,1) → (5,1) → (5,2) → (5,3) → (5,4) → (5,5). **Total cost = 8.** Nodes expanded = 9 (the goal is popped on step 10); maximum stack size = 4.

Note: DFS reached an optimal path here only because of the chosen order. If the order is changed to Right, Left, Down, Up, DFS returns the longer path (1,1) → (1,2) → (1,3) → (1,4) → (1,5) → (2,5) → (2,4) → (3,4) → (3,5) → (4,5) → (5,5) with cost 10. DFS gives no guarantee of the shortest path.

Input:

dfs program.py - C:\Users\Anusha\AppData\Local\Programs\Python\Python311\dfs program.py (3.11.2)

File Edit Format Run Options Window Help

```
start = (1, 1)
goal = (5, 5)

blocked = {
    (2, 2), (2, 3), (3, 3),
    (4, 2), (4, 4)
}

moves = [
    (-1, 0),    # Up
    (1, 0),     # Down
    (0, -1),    # Left
    (0, 1)      # Right
]

stack = []
stack.append((start, [start]))

visited = {start}

while stack:

    current, path = stack.pop()

    print("Expanded:", current)
    print("Stack:", [x[0] for x in stack])

    if current == goal:
        print("\nGoal Reached!")
        print("DFS Path:", path)
        print("Total Cost:", len(path) - 1)
        break

    # Reverse order so that Up is explored first
    for move in reversed(moves):

        new_row = current[0] + move[0]
        new_col = current[1] + move[1]

        new_position = (new_row, new_col)
```

dfs program.py - C:\Users\Anusha\AppData\Local\Programs\Python\Python311\dfs program.py (3.11.2)

File Edit Format Run Options Window Help

```
    (0, -1),    # Left
    (0, 1)      # Right
]

stack = []
stack.append((start, [start]))

visited = {start}

while stack:

    current, path = stack.pop()

    print("Expanded:", current)
    print("Stack:", [x[0] for x in stack])

    if current == goal:
        print("\nGoal Reached!")
        print("DFS Path:", path)
        print("Total Cost:", len(path) - 1)
        break

    # Reverse order so that Up is explored first
    for move in reversed(moves):

        new_row = current[0] + move[0]
        new_col = current[1] + move[1]

        new_position = (new_row, new_col)

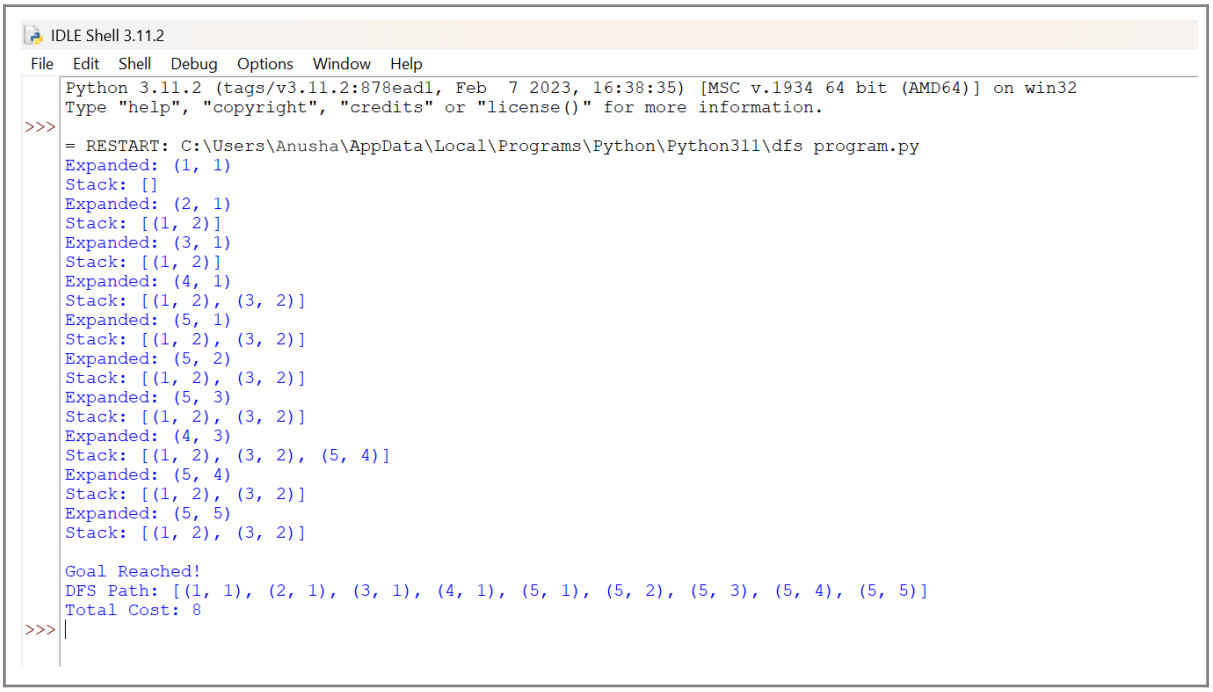
        if (1 <= new_row <= 5 and
            1 <= new_col <= 5 and
            new_position not in blocked and
            new_position not in visited):

            visited.add(new_position)

            new_path = path + [new_position]

            stack.append((new_position, new_path))
```

output:


IDLE Shell 3.11.2
File Edit Shell Debug Options Window Help
Python 3.11.2 (tags/v3.11.2:878ead1, Feb 7 2023, 16:38:35) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Anusha\AppData\Local\Programs\Python\Python311\dfs program.py
Expanded: (1, 1)
Stack: []
Expanded: (2, 1)
Stack: [(1, 2)]
Expanded: (3, 1)
Stack: [(1, 2)]
Expanded: (4, 1)
Stack: [(1, 2), (3, 2)]
Expanded: (5, 1)
Stack: [(1, 2), (3, 2)]
Expanded: (5, 2)
Stack: [(1, 2), (3, 2)]
Expanded: (5, 3)
Stack: [(1, 2), (3, 2)]
Expanded: (4, 3)
Stack: [(1, 2), (3, 2), (5, 4)]
Expanded: (5, 4)
Stack: [(1, 2), (3, 2)]
Expanded: (5, 5)
Stack: [(1, 2), (3, 2)]
Goal Reached!
DFS Path: [(1, 1), (2, 1), (3, 1), (4, 1), (5, 1), (5, 2), (5, 3), (5, 4), (5, 5)]
Total Cost: 8
>>>

f. Comparison of BFS and DFS

Let b = branching factor (at most 4, effectively about 2–3 here), d = depth of the shallowest goal (8), m = maximum depth of the search tree, and V = number of states (20).

Criterion	BFS	DFS
Completeness	Complete: finite state space and finite branching factor, so it always finds a solution if one exists.	Complete only with an explored set (cycle checking) in a finite space. Plain tree search can loop forever, e.g. (1,1) ↔ (2,1).
Optimality	Optimal, because every step costs 1: it finds the shallowest goal, which is the cheapest path (cost 8).	Not optimal: returns the first path found, which can be longer (cost 10 with a different move order).
Time complexity	O(b^d) for tree search, O(V + E) for graph search. Here: 18 nodes expanded.	O(b^m) for tree search, O(V + E) for graph search. Here: 9 nodes expanded (fast because of a favourable order).
Space complexity	O(b^d): the whole frontier is stored. Here: maximum frontier size 4.	O(b·m) for tree search with path checking; O(V) with an explored set. Here: maximum stack size 4.

Conclusion: BFS is more suitable for this warehouse delivery problem.

- The robot should use the fewest moves (least time and energy). BFS guarantees the minimum cost of 8 for every move ordering, whereas DFS can return a path of cost 10 or more.
- The grid has only 20 states, so BFS's larger memory use is not a concern. DFS's main advantage (small memory) does not matter here.
- For much larger warehouses, memory and time would grow, and an informed search such as A* with the Manhattan-distance heuristic (admissible here) would be the natural upgrade while keeping optimality.

  Input:

comparision.py - C:/Users/Anusha/AppData/Local/Programs/Python/Python311/comparision.py (3.11.2)

File Edit Format Run Options Window Help

```
from collections import deque

# 5x5 warehouse
grid = [
    [0, 0, 0, 0, 0],
    [0, 1, 1, 0, 0],
    [0, 0, 1, 0, 0],
    [0, 1, 0, 1, 0],
    [0, 0, 0, 0, 0]
]

start = (0, 0)
goal = (4, 4)

# Up, Down, Left, Right
directions = [
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1)
]

def bfs():
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        current, path = queue.popleft()

        if current == goal:
            return path

        r, c = current

        for dr, dc in directions:
            nr = r + dr
            nc = c + dc

            if (0 <= nr < 5 and
                0 <= nc < 5 and
                grid[nr][nc] == 0 and
```

comparision.py - C:/Users/Anusha/AppData/Local/Programs/Python/Python311/comparision.py (3.11.2)

File Edit Format Run Options Window Help

```
    (-1, 0),
    (1, 0),
    (0, -1),
    (0, 1)
]

def bfs():
    queue = deque([(start, [start])])
    visited = {start}

    while queue:
        current, path = queue.popleft()

        if current == goal:
            return path

        r, c = current

        for dr, dc in directions:
            nr = r + dr
            nc = c + dc

            if (0 <= nr < 5 and
                0 <= nc < 5 and
                grid[nr][nc] == 0 and
                (nr, nc) not in visited):

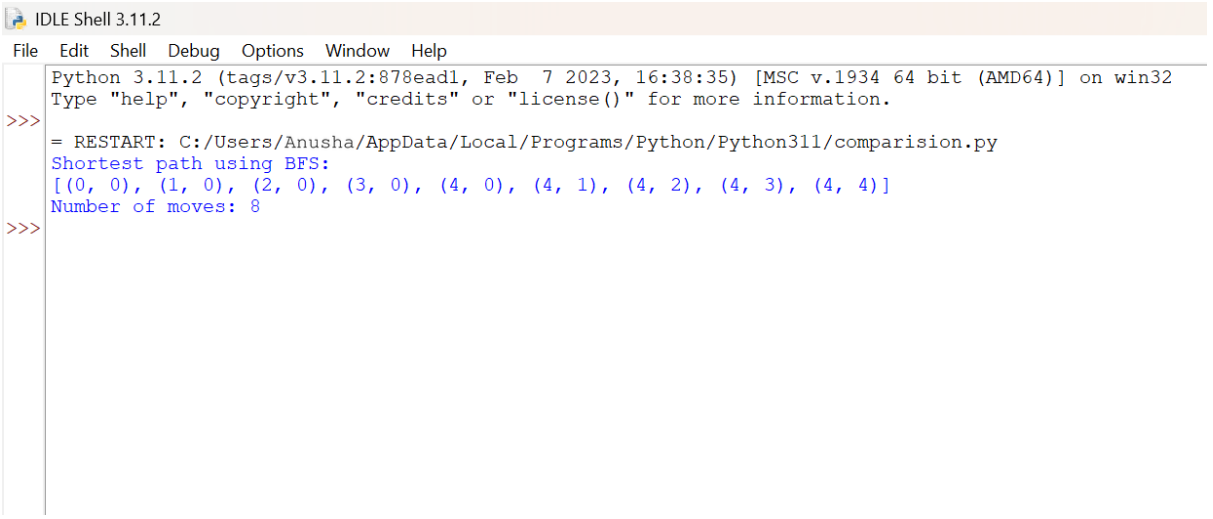
                visited.add((nr, nc))
                queue.append(((nr, nc), path + [(nr, nc)]))

    return None


path = bfs()

if path:
    print("Shortest path using BFS:")
    print(path)
    print("Number of moves:", len(path) - 1)
else:
```

Output:


IDLE Shell 3.11.2
File Edit Shell Debug Options Window Help
Python 3.11.2 (tags/v3.11.2:878ead1, Feb 7 2023, 16:38:35) [MSC v.1934 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:/Users/Anusha/AppData/Local/Programs/Python/Python311/comparision.py
Shortest path using BFS:
[(0, 0), (1, 0), (2, 0), (3, 0), (4, 0), (4, 1), (4, 2), (4, 3), (4, 4)]
Number of moves: 8
>>>